

claude-windows / f81fb2b5-e34b-427e-a8a1-a37eb4539a49

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\f81fb2b5-e34b-427e-a8a1-a37eb4539a49\subagents\workflows\wf\_085a5118-2cb\agent-a156ae363abceff73.jsonl`
- SHA-256: `614afffc46fcff9e9cac9d4ba425fadd255a12cec8d51eafa29cfa85eeb01f26`
- Source modified: `2026-06-20T16:57:44+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf\_085a5118-2cb`
- session_id: `f81fb2b5-e34b-427e-a8a1-a37eb4539a49`

Transcript

1. user (2026-06-20T16:56:24.755Z)

Map EVERY per-video artifact write site in the WORKER + STORAGE + BUCKET layout. Read backend/worker/__main__.py (cmd_infer AND cmd_train) and backend/storage/*.py. Find: the scratch temp dirs (labels/trajectories/videos/artifacts), the bucket output prefixes (outputs/<video_id>/..., weights/<version>/..., labels/, videos/, parity/), what storage.put/fetch/list_uris write where, and how video_id (MD5) keys outputs. Record exact file:line, path/prefix pattern, hardcoded?, keyed_by, and config/env knobs (scratch, out-uri, corpus-uri).

2. assistant (2026-06-20T16:56:27.088Z)

I'll help you map all per-video artifact write sites in the WORKER + STORAGE + BUCKET layout. Let me start by exploring the codebase structure and reading the key files.

3. user (2026-06-20T16:56:27.510Z)

```
backend\worker\__main__.py
```

4. user (2026-06-20T16:56:27.785Z)

```
backend\storage\gcs.py
backend\storage\local.py
backend\storage\s3.py
backend\storage\__init__.py
backend\storage\base.py
backend\storage\ref.py
backend\storage\gdrive.py
```

5. user (2026-06-20T16:56:27.899Z)

```
backend\eval\__init__.py
backend\eval\classifier.py
backend\eval\training.py
backend\providers\__init__.py
backend\sports\__init__.py
backend\sports\badminton.py
backend\sports\base.py
backend\utils\__init__.py
backend\utils\geometry.py
backend\annotations\base.py
backend\annotations\file_provider.py
backend\annotations\automated_provider.py
backend\annotations\__init__.py
backend\tools\rally_slicer.py
```

backend\tools__init__.py
backend\eval\gemini_refine.py
backend\eval\audio__init__.py
backend\eval\audio\e1_probe.py
backend\eval\export_wasb_segments.py
backend\pipeline\audio__init__.py
backend\pipeline\validators__init__.py
backend\eval\metrics.py
backend\eval\regression.py
backend\eval\annotations.py
backend\features__init__.py
backend\reporting\harvest.py
backend\reporting\run_signals.py
backend\eval\calibrate_wasb.py
backend\eval\calibration.py
backend\eval\eval_partial_labels.py
backend\eval\rally_seq_proto.py
backend\features\rally_seq.py
backend\pipeline\detectors__init__.py
backend\pipeline\segmenters\base.py
backend\pipeline\validators\blur.py
backend\pipeline\validators\format.py
backend\pipeline\validators\pts_continuity.py
backend\pipeline\validators\relevance.py
backend\pipeline\validators\resolution_fps.py
backend\pipeline\validators\scene_cut.py
backend\providers\base.py
backend\reporting__init__.py
backend\eval\calibrate_local.py
backend\eval\gt_loader.py
backend\eval\harness.py
backend\pipeline\audio\hit_detector.py
backend\pipeline\validators\base.py
backend\utils\paths.py
backend\utils\video.py
backend\pipeline\segmenters\tracknet_hybrid.py
backend\pipeline\detectors\fusion_features.py
backend\pipeline\detectors\rally_gate.py
backend\pipeline\segmenters__init__.py
backend\tools\export.py
backend\utils\logging_setup.py
backend\pipeline\detectors\audio_features.py
backend\eval\eval_stats.py
backend\eval\score_calibration.py
backend\pipeline\detectors\person_detector.py
backend\infrastructure\execution_policy.py
backend\utils\run_telemetry.py
backend\eval\batch.py
backend\utils\hashing.py
backend\eval\fusion_golden.py
backend\eval\per_user_gate.py
backend\eval\per_user_eval.py
backend\eval\per_user_regression.py
backend\pipeline\personalization\serving.py
backend\pipeline\personalization__init__.py
backend\eval\served_gate_a.py
backend\eval\distill_local.py
backend\eval\promotion.py
backend\eval\serve_contrast.py
backend\eval\splits.py
backend\pipeline\detectors\base.py
backend\pipeline\detectors\wasb_runner.py
backend\eval\segmentation_metrics.py
backend\eval\tolerance_metrics.py
backend\eval>windowing.py

```

backend\results.py
backend\api\__init__.py
backend\api\uploads.py
backend\config\loader.py
backend\eval\ablation_pdf.py
backend\pipeline\detectors\tracknet_runner.py
backend\api\job_worker.py
backend\eval\ablation.py
backend\eval\experiment.py
backend\eval\fusion_audio.py
backend\eval\fusion_compare.py
backend\eval\golden_fixtures.py
backend\eval\golden_real_fixtures.py
backend\infrastructure\database.py
backend\main.py
backend\pipeline\segmenters\motion.py
backend\providers\gemini.py
backend\pipeline\detectors\court_gate.py
backend\eval\rally_seg_eval.py
backend\config\__init__.py
backend\pipeline\detectors\stub_runner.py
(Results are truncated. Consider using a more specific path or pattern.)

```

6. assistant (2026-06-20T16:56:29.707Z)

Now let me read the main worker file:

7. user (2026-06-20T16:56:30.168Z)

```

1      """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3      python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4          [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5      python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7      The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
8      ``local:`` is an identity passthrough; ``s3:`` / ``gcs:`` is an object store), so existing
9      local runs
10     are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
11     ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
12     key) ?
13     the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
14     This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
15     CLI uses
16     (validator registry ? segmenter ? report), never a forked pipeline.
17     """
18     from __future__ import annotations
19
20     import argparse
21     import csv
22     import json
23     import os
24     import sys
25     import tempfile
26     from typing import Any, List
27
28     from backend import storage
29     from backend.config import load_config
30     from backend.infrastructure.database import Database
31     from backend.pipeline.segmenters.base import segmenter_registry
32     from backend.pipeline.validators.base import registry as validator_registry
33     from backend.reporting import emit_indexer_report
34     from backend.results import Failure

```

```

33     from backend.utils.hashing import compute_video_id
34
35
36     def _coerce(v: str) -> Any:
37         """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
38         str."""
39         low = v.lower()
40         if low in ("true", "false"):
41             return low == "true"
42         for cast in (int, float):
43             try:
44                 return cast(v)
45             except ValueError:
46                 pass
47         return v
48
49     def _apply_overrides(config: Any, sets: List[str]) -> None:
50         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
51         indexing.skip_ai_handoff=true)."""
52         for kv in sets or []:
53             key, sep, val = kv.partition("=")
54             if not sep:
55                 raise ValueError(f"--set expects k=v, got {kv!r}")
56             parts = key.split(".")
57             obj = config
58             for p in parts[:-1]:
59                 obj = getattr(obj, p)
60             setattr(obj, parts[-1], _coerce(val))
61
62     def _write_intervals_csv(segments: List[dict], path: str) -> None:
63         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
64         friendly
65         artifact (same schema as the rally-annotator labels)."""
66         with open(path, "w", newline="") as f:
67             w = csv.writer(f)
68             w.writerow(["start", "end", "shot_count", "ending_reason"])
69             for s in segments:
70                 w.writerow([
71                     f'{float(s.get("start_time", 0.0)):.3f}',
72                     f'{float(s.get("end_time", 0.0)):.3f}',
73                     s.get("shot_count", ""),
74                     s.get("ending_reason", s.get("shot_count_confidence", "")),
75                 ])
76
77     def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
78     None:
79         with open(path, "w") as f:
80             json.dump({
81                 "video_id": video_id,
82                 "status": status,
83                 "segment_count": len(segments),
84                 "segments": [{"start": float(s.get("start_time", 0.0)),
85                             "end": float(s.get("end_time", 0.0))} for s in segments],
86             }, f, indent=2)
87
88     def cmd_infer(args: argparse.Namespace) -> int:
89         config = load_config(args.config) if args.config else load_config()
90         _apply_overrides(config, args.set)
91         storage_cfg = config.storage.model_dump()
92
93         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")

```

```

94         os.makedirs(scratch, exist_ok=True)
95         config.output.output_dir = scratch
96
97         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
98         local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
99         print(f"[worker] pulled {args.video_uri} -> {local_video}")
100
101         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
102         video_id = compute_video_id(local_video)
103
104         # 3. VALIDATE (same registry as the main CLI).
105         val_results, val_context = validator_registry.run_all_with_context(local_video,
config)
106         failures = [r for r in val_results if not r.passed]
107         db = Database(os.path.join(scratch, config.output.db_name))
108         db.add_video(video_id, local_video, "failed" if failures else "processed",
[r.model_dump() for r in val_results])
109
110
111         segments: List[dict] = []
112         segmenter_actual = None
113         segmentation_ran = False
114         status = "failed"
115         try:
116             if failures:
117                 print("[worker] validation FAILED: "
+ "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
118                 return 2
119             seg_name = args.segmenter or config.indexing.default_segmenter
120             segmenter_cls = segmenter_registry.get(seg_name)
121             if not segmenter_cls:
122                 print(f"[worker] unknown segmenter: {seg_name!r} "
f"(have {list(segmenter_registry.list_available())})")
123                 return 2
124             segmenter_actual = seg_name
125             result = segmenter_cls(db, config).process_video(video_id, local_video,
max_frames=args.max_frames)
126             segmentation_ran = True
127             if isinstance(result, Failure):
128                 print(f"[worker] segmenter FAILED: {result.message}")
129                 return 3
130             segments = result.value if hasattr(result, "value") else result
131             status = "success"
132         finally:
133             # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
134             emit_indexer_report(
135                 db=db, video_id=video_id, video_path=local_video, config=config,
136                 val_results=val_results, val_context=val_context,
137                 segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
138                 was_reingest=False, segmentation_ran=segmentation_ran,
139             )
140
141
142
143         # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
144         out_local = os.path.join(scratch, "outputs", video_id)
145         os.makedirs(out_local, exist_ok=True)
146         _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
147         _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))
148
149         out_prefix = args.out_uri.rstrip("/")
150         pushed = []
151         for fn in sorted(os.listdir(out_local)):

```

```

152         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
153         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
154         pushed.append(uri)
155
156     print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
157     for u in pushed:
158         print("    ", u)
159     return 0
160
161
162     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
163     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
164     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
165
166
167     def _probe_video_fps_width(path: str):
168         """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
169
170         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
171         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
172         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
173         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
174         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
175         rather than guesses.
176         """
177         import json
178         import subprocess
179         try:
180             out = subprocess.check_output(
181                 ["ffprobe", "-v", "error", "-select_streams", "v:0",
182                  "-show_entries", "stream=r_frame_rate,width", "-of", "json", path],
183                 stderr=subprocess.DEVNULL).decode()
184             st = (json.loads(out).get("streams") or [{}])[0]
185             num, _ , den = str(st.get("r_frame_rate", "0/1")).partition("/")
186             fps = float(num) / float(den or "1")
187             width = float(st.get("width") or 0)
188             if fps > 0 and width > 0:
189                 return fps, width
190         except (subprocess.SubprocessError, ValueError, KeyError, OSError,
json.JSONDecodeError):
191             pass
192         return None
193
194
195     def _resolve_train_detector(args: argparse.Namespace, config: Any):
196         """Build the trajectory DetectorRunner for ``--trajectory-source detector``, honouring
197         detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
198         stub=CPU mock). Returns the runner, or prints an error + returns None.
199
200         Reuses the segmenter's ``_resolve_runner`` seam so the worker and the live CLI build
the
201         detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
202         no shuttle detector and is rejected.
203         """
204         from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter

```

```

205
206     seg_cls = segmenter_registry.get(args.segmenter)
207     if not seg_cls:
208         print(f"[worker] unknown segmenter: {args.segmenter!r} "
209               f"(have {list(segmenter_registry.list_available())})")
210         return None
211     seg = seg_cls(None, config)
212     if not isinstance(seg, TrajectoryHybridSegmenter):
213         print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
214               f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid /
fusion_hybrid).")
215         return None
216     try:
217         runner, _ = seg._resolve_runner(config.get("indexing", {}))
218     except NotImplementedError as e:
219         # e.g. detector_impl='native' for a family without a native runner (tracknet, or
220         # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
traceback.
221         print(f"[worker] detector not available: {e}")
222         return None
223     ok, msg = runner.healthcheck()
224     if not ok:
225         print(f"[worker] detector environment not ready: {msg}")
226         return None
227     print(f"[worker] detector ready ({args.segmenter}): {msg}")
228     return runner
229
230
231 def cmd_train(args: argparse.Namespace) -> int:
232     """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
233     to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.
234
235     Two trajectory sources (the train pipeline + harness are identical either way):
236     * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
237         shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
238     * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
240         is the M3.5 "first real training" path; only the trajectory source flips.
241     """
242     import subprocess
243
244     config = load_config(args.config) if args.config else load_config()
245     _apply_overrides(config, args.set)
246     storage_cfg = config.storage.model_dump()
247
248     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
249     labels_dir = os.path.join(scratch, "labels")
250     traj_dir = os.path.join(scratch, "trajectories")
251     videos_dir = os.path.join(scratch, "videos")
252     os.makedirs(labels_dir, exist_ok=True)
253     os.makedirs(traj_dir, exist_ok=True)
254
255     corpus = args.corpus_uri.rstrip("/")
256     label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/",
config=storage_cfg)
257                        if u.lower().endswith(".csv"))
258     if len(label_uris) < 2:
259         print(f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
260               f"under {corpus}/labels/")

```

```

261         return 2
262
263     # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
264     runner = None
265     video_by_stem: dict = {}
266     if args.trajectory_source == "detector":
267         os.makedirs(videos_dir, exist_ok=True)
268         runner = _resolve_train_detector(args, config)
269         if runner is None:
270             return 2
271         for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
272             vname = storage.parse_uri(vuri).name
273             if not vname.lower().endswith(_VIDEO_EXTS):
274                 continue # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
275             video_by_stem[os.path.splitext(vname)[0]] = vuri
276
277     print(f"[worker] trajectory source: {args.trajectory_source}")
278     specs: List[dict] = []
279     for uri in label_uris:
280         fname = storage.parse_uri(uri).name # e.g.
GX020094_proxy.rallies.csv
281         name = fname.replace(".rallies.csv", "").replace(".csv", "")
282         local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
283         if args.trajectory_source == "detector":
284             vuri = video_by_stem.get(name)
285             if not vuri:
286                 print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
287                 continue
288             local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
289                                     config=storage_cfg)
290             video_id = compute_video_id(local_video)
291             # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
292             # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
293             meta = _probe_video_fps_width(local_video)
294             if meta is None:
295                 print(f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess).")
296                 continue
297             fps, frame_width = meta
298             traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
299             if not traj:
300                 print(f"[worker] detector produced no trajectory for {name!r} ?
skipping.")
301                 continue
302             specs.append({"name": name, "trajectory": traj, "labels": local_label,
303                          "fps": fps, "frame_width": frame_width})
304         else:
305             from backend.pipeline.detectors.stub_runner import
synth_trajectory_from_labels
306             traj = os.path.join(traj_dir, f"{name}_traj.csv")
307             synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
308             specs.append({"name": name, "trajectory": traj, "labels": local_label,
309                          "fps": args.fps, "frame_width": args.frame_width})
310
311     if len(specs) < 2:
312         print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")

```

```

313         return 2
314     manifest_path = os.path.join(scratch, "manifest.json")
315     with open(manifest_path, "w", encoding="utf-8") as f:
316         json.dump(specs, f, indent=2)
317     src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock
stub"
318     print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
319
320     out_dir = os.path.join(scratch, "artifacts")
321     cmd = [sys.executable, "-m", "training.gen0.harness",
322           "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
323     if args.floor:
324         floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
config=storage_cfg)
325                        if "://" in args.floor else args.floor)
326         cmd += ["--floor", floor_local]
327     print("[worker] training:", " ".join(cmd))
328     rc = subprocess.run(cmd).returncode
329     if rc != 0:
330         print(f"[worker] gen0 harness failed (rc={rc})")
331         return rc
332
333     # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
334     bundle = os.path.join(out_dir, args.version)
335     out_prefix = args.out_uri.rstrip("/")
336     pushed = []
337     for fn in sorted(os.listdir(bundle)):
338         uri = f"{out_prefix}/weights/{args.version}/{fn}"
339         storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
340         pushed.append(uri)
341     print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
342     for u in pushed:
343         print("    ", u)
344     return 0
345
346
347 def build_parser() -> argparse.ArgumentParser:
348     p = argparse.ArgumentParser(prog="python -m backend.worker",
349                                description="Storage-aware worker: pull ? run pipeline ?
push.")
350     sub = p.add_subparsers(dest="cmd", required=True)
351
352     pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
353     pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// /
gcs://")
354     pi.add_argument("--out-uri", required=True, help="output prefix
(outputs/<video_id>/? is appended)")
355     pi.add_argument("--segmenter", default=None, help="default:
indexing.default_segmenter")
356     pi.add_argument("--config", default=None, help="config.json path (default:
./config.json)")
357     pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp
dir)")
358     pi.add_argument("--max-frames", type=int, default=None)
359     pi.add_argument("--set", action="append", default=[], metavar="k=v",
360                    help="config override, e.g. --set indexing.skip_ai_handoff=true")
361     pi.set_defaults(func=cmd_infer)
362
363     pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] ->
trajectories -> harness)")
364     pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+
videos/ for "
365                   "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
366     pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/
is appended)")

```

```

367         pt.add_argument("--version", required=True, help="semver for the artifact bundle")
368         pt.add_argument("--trajectory-source", choices=["synth", "detector"],
default="synth",
369                             help="synth = CPU-mock label-consistent trajectories (default, no
GPU); "
370                             "detector = REAL shuttle trajectories via the resolved
DetectorRunner "
371                             "(needs videos/ in the corpus; honours detector_impl).")
372         pt.add_argument("--segmenter", default="wasb_hybrid",
373                             help="trajectory hybrid whose detector is used for --trajectory-
source detector")
374         pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or
storage URI)")
375         pt.add_argument("--config", default=None)
376         pt.add_argument("--scratch", default=None)
377         pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps
(detector probes the video)")
378         pt.add_argument("--frame-width", type=float, default=1280.0,
379                             help="synth trajectory frame width (detector probes the video)")
380         pt.add_argument("--set", action="append", default=[], metavar="k=v")
381         pt.set_defaults(func=cmd_train)
382         return p
383
384
385     def main(argv=None) -> int:
386         args = build_parser().parse_args(argv)
387         return args.func(args)
388
389
390     if __name__ == "__main__":
391         sys.exit(main())
392

```

8. user (2026-06-20T16:56:30.377Z)

```

1     """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3     URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4     lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
5     ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and
preserves
6     today's behaviour byte-for-byte; ``gdrive://`` likewise needs none ? it reads a locally
MOUNTED
7     Google Drive (Google Drive for Desktop) as plain files.
8
9     Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
10    ``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an
identity
11    passthrough. See ``docs/DECOUPLED_COMPUTE``.
12    """
13    from __future__ import annotations
14
15    import os
16    from typing import Any, Dict, Iterator, Optional
17
18    from backend.storage.base import StorageBackend
19    from backend.storage.ref import StorageRef, StorageStat, parse_uri
20
21    __all__ = [
22        "StorageBackend", "StorageRef", "StorageStat", "parse_uri",
23        "get_storage", "fetch", "put", "list_uris",
24    ]
25
26

```

```

27     def _backend_class(name: str):
28         """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
29         if name == "local":
30             from backend.storage.local import LocalStorage
31             return LocalStorage
32         if name == "s3":
33             from backend.storage.s3 import S3Storage
34             return S3Storage
35         if name == "gcs":
36             from backend.storage.gcs import GCSStorage
37             return GCSStorage
38         if name == "gdrive":
39             from backend.storage.gdrive import GDriveStorage
40             return GDriveStorage
41         raise ValueError(f"unknown storage backend: {name!r} (expected
local|s3|gcs|gdrive)")
42
43
44     def get_storage(backend: Optional[str] = None, *,
45                     config: Optional[Dict[str, Any]] = None, **overrides) -> StorageBackend:
46         """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
47         the implementation; its per-backend settings come from ``config[backend]`` merged
with
48         ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
49         cfg = config or {}
50         name = backend or cfg.get("backend", "local")
51         settings: Dict[str, Any] = dict(cfg.get(name, {}) or {})
52         settings.update(overrides)
53         return _backend_class(name)(**settings)
54
55
56     def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None, *,
57              config: Optional[Dict[str, Any]] = None, overwrite: bool = False) -> str:
58         """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
59         (no copy); for s3/gcs it downloads to ``dst``."""
60         ref = parse_uri(uri)
61         return get_storage(ref.backend, config=config).fetch_to_local(ref, dst,
overwrite=overwrite)
62
63
64     def put(local_path: "str | os.PathLike[str]", uri: str, *,
65            config: Optional[Dict[str, Any]] = None, content_type: Optional[str] = None) ->
str:
66         """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
67         ref = parse_uri(uri)
68         get_storage(ref.backend, config=config).put(local_path, ref,
content_type=content_type)
69         return uri
70
71
72     def list_uris(prefix_uri: str, *, config: Optional[Dict[str, Any]] = None) ->
Iterator[str]:
73         """Yield the URI of every object under ``prefix_uri``."""
74         ref = parse_uri(prefix_uri)
75         for r in get_storage(ref.backend, config=config).list(ref):
76             yield r.uri
77

```

9. user (2026-06-20T16:56:30.522Z)

```

1     """The ``StorageBackend`` ABC ? one interface for local + object-store + Drive.
2

```

```

3      Mirrors the codebase's ABC+Registry idiom (cf. ``SegmenterRegistry``). The bucket
backends
4      live in sibling modules and lazy-import their cloud SDK so ``import backend.storage``
never
5      pulls ``boto3`` / ``google-cloud-storage`` (``gdrive`` is pure local-FS over a mounted
Drive ?
6      no SDK). See ``PLATFORM_ARCHITECTURE.md`` §3.1 and ``docs/DECOUPLED_COMPUTE/``.
7      ""
8      from __future__ import annotations
9
10     import os
11     from abc import ABC, abstractmethod
12     from typing import Iterator
13
14     from backend.storage.ref import StorageRef, StorageStat
15
16
17     class StorageBackend(ABC):
18         """Backend-agnostic object storage. ``fetch_to_local`` + ``put`` are the two the
19         worker dispatcher needs; the rest support listing/checkpointing/serving."""
20
21         @abstractmethod
22         def fetch_to_local(self, ref: StorageRef, dst: "str | os.PathLike[str] | None" =
None,
23                             *, overwrite: bool = False) -> str:
24             """Make the object available as a local file; return its local path. For the
25             ``local`` backend this is an identity passthrough (the object is already
local)."""
26
27         @abstractmethod
28         def put(self, local_path: "str | os.PathLike[str]", ref: StorageRef, *,
29                 content_type: "str | None" = None) -> StorageRef:
30             """Upload/copy a local file to ``ref``; return ``ref``."""
31
32         @abstractmethod
33         def exists(self, ref: StorageRef) -> bool: ...
34
35         @abstractmethod
36         def stat(self, ref: StorageRef) -> StorageStat: ...
37
38         @abstractmethod
39         def list(self, prefix: StorageRef) -> Iterator[StorageRef]:
40             """Yield a ``StorageRef`` per object under ``prefix``."""
41
42         @abstractmethod
43         def delete(self, ref: StorageRef) -> None: ...
44
45         def signed_url(self, ref: StorageRef, *, ttl: int = 3600, method: str = "GET") ->
str:
46             """A time-limited URL handing the bytes to a browser/LLM. Not every backend can
47             sign (``local`` cannot; Drive only emulates) ? callers must treat
48             ``NotImplementedError`` as expected and fall back to ``fetch_to_local``."""
49             raise NotImplementedError(f"{type(self).__name__} cannot produce signed URLs")
50

```

10. assistant (2026-06-20T16:56:32.261Z)

Now let me read the storage implementations:

11. user (2026-06-20T16:56:32.675Z)

```

1      """Local-filesystem storage backend ? the default, zero new dependencies.
2
3      Wraps today's behaviour byte-for-byte: ``fetch_to_local`` is an identity passthrough
(the

```

```

4     file is already local), ``put`` is a copy (a no-op when src == dst). With
    ``backend='local'``
5     the whole pipeline behaves exactly as before the storage layer existed.
6     """
7     from __future__ import annotations
8
9     import os
10    import shutil
11    from datetime import datetime
12    from typing import Iterator
13
14    from backend.storage.base import StorageBackend
15    from backend.storage.ref import StorageRef, StorageStat
16
17
18    class LocalStorage(StorageBackend):
19        def __init__(self, *, base_dir: str = "", **_ignored) -> None:
20            # Relative keys resolve under base_dir when set; absolute keys are used as-is.
21            self.base_dir = base_dir
22
23        def _path(self, ref: StorageRef) -> str:
24            p = ref.key
25            if self.base_dir and not os.path.isabs(p):
26                return os.path.join(self.base_dir, p)
27            return p
28
29        def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
30            # The object is already on the local FS ? return its path (identity). ``dst`` is
31            # honoured only if it names a *different* location (a copy is then made).
32            src = self._path(ref)
33            if dst is None or os.path.abspath(str(dst)) == os.path.abspath(src):
34                return src
35            dst = str(dst)
36            if os.path.exists(dst) and not overwrite:
37                return dst
38            os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
39            shutil.copyfile(src, dst)
40            return dst
41
42        def put(self, local_path, ref, *, content_type=None) -> StorageRef:
43            dst = self._path(ref)
44            if os.path.abspath(str(local_path)) != os.path.abspath(dst):
45                os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
46                shutil.copyfile(str(local_path), dst)
47            return ref
48
49        def exists(self, ref) -> bool:
50            return os.path.exists(self._path(ref))
51
52        def stat(self, ref) -> StorageStat:
53            st = os.stat(self._path(ref))
54            return StorageStat(size=st.st_size,
55modified=datetime.fromtimestamp(st.st_mtime))
56
57        def list(self, prefix) -> Iterator[StorageRef]:
58            base = self._path(prefix)
59            if os.path.isdir(base):
60                for root, _dirs, files in os.walk(base):
61                    for fn in sorted(files):
62                        full = os.path.join(root, fn)
63                        yield StorageRef("local", "", full, f"local://{full}")
64            elif os.path.exists(base):
65                yield StorageRef("local", "", base, f"local://{base}")
66
67        def delete(self, ref) -> None:

```

```

67         path = self._path(ref)
68         if os.path.exists(path):
69             os.remove(path)
70

```

12. user (2026-06-20T16:56:32.945Z)

```

1     """Storage value objects + URI parsing for the pluggable storage layer.
2
3     A ``StorageRef`` is a backend-agnostic pointer to one object. URIs:
4
5         <bare path>                -> local    (today's behaviour; e.g. C:/v/x.mp4,
/data/x.mp4)
6         local://<path>             -> local    (local:///abs or local://rel/p)
7         s3://<bucket>/<key>        -> s3       (AWS + R2 + DO Spaces + MinIO via
endpoint_url)
8         gcs://<bucket>/<key>      -> gcs      (gs:// accepted, normalized to gcs://)
9         gdrive://<path>           -> gdrive   (path under your locally-MOUNTED Google
Drive "My Drive")
10
11     No cloud SDKs are imported here ? this module is pure stdlib so ``import
backend.storage``
12     stays cheap.
13     """
14     from __future__ import annotations
15
16     import re
17     from dataclasses import dataclass
18     from datetime import datetime
19     from typing import Optional
20
21     _SCHEME_RE = re.compile(r"^[A-Za-z][A-Za-z0-9+.\-]*://")
22     _KNOWN = ("local", "s3", "gcs", "gdrive")
23
24
25     @dataclass(frozen=True)
26     class StorageStat:
27         """Lightweight object metadata (uniform across backends)."""
28         size: int
29         modified: Optional[datetime] = None
30         etag: Optional[str] = None
31         content_type: Optional[str] = None
32
33
34     @dataclass(frozen=True)
35     class StorageRef:
36         """A backend + a location. ``bucket`` is the S3/GCS bucket; for ``local`` and
``gdrive`` it is
37         empty and ``key`` carries the path (a filesystem path, or a path under the mounted
Drive)."""
38         backend: str
39         bucket: str
40         key: str
41         uri: str
42
43         @property
44         def name(self) -> str:
45             return self.key.rstrip("/").rsplit("/", 1)[-1]
46
47
48     def parse_uri(uri: str) -> StorageRef:
49         """Parse a storage URI (or a bare filesystem path) into a ``StorageRef``.
50
51         A string with no ``scheme://`` prefix (including Windows ``C:\\...`` paths, which
have no ``//``) is treated as a local filesystem path ? preserving today's

```

```

behaviour.
53         """
54         s = str(uri)
55         m = _SCHEME_RE.match(s)
56         if not m:
57             return StorageRef("local", "", s, f"local://{s}")
58         scheme = m.group(1).lower()
59         rest = s[m.end():]
60         if scheme == "gs":
61             scheme = "gcs"
62         if scheme in ("local", "gdrive"):
63             # Path-keyed, no bucket: local FS, or a path under the mounted Google Drive "My
Drive".
64             return StorageRef(scheme, "", rest, s)
65         if scheme in _KNOWN:
66             bucket, _, key = rest.partition("/")
67             return StorageRef(scheme, bucket, key, s)
68         raise ValueError(f"unsupported storage URI scheme: {scheme}://{ (in {s!r})}")
69

```

13. user (2026-06-20T16:56:33.139Z)

```

1         """S3 (and S3-compatible) storage backend ? first-class object buckets.
2
3         One backend covers **AWS S3, Cloudflare R2, DigitalOcean Spaces, and MinIO** ? they
differ
4         only by ``endpoint_url`` (+ region / addressing style). Uses ``boto3``, imported lazily
so
5         ``import backend.storage`` never pulls it; install with the ``storage-s3`` extra.
6
7         Credentials are NEVER read from config ? only ``endpoint_url`` / ``region`` / addressing
8         style. boto3's default chain supplies creds (``AWS_ACCESS_KEY_ID`` /
``AWS_SECRET_ACCESS_KEY``
9         / ``AWS_SESSION_TOKEN`` env, instance role, or ``~/.aws``). A ``client`` may be injected
10        (tests pass a fake; production lets the constructor build one via boto3).
11        """
12        from __future__ import annotations
13
14        import os
15        from typing import Any, Iterator, Optional
16
17        from backend.storage.base import StorageBackend
18        from backend.storage.ref import StorageRef, StorageStat
19
20
21        class S3Storage(StorageBackend):
22            def __init__(self, *, client: Any = None, endpoint_url: Optional[str] = None,
23                region: Optional[str] = None, addressing_style: str = "auto",
24                signed_url_ttl: int = 3600, **_ignored) -> None:
25                self._ttl = signed_url_ttl
26                if client is not None:
27                    self._client = client
28                return
29            import boto3 # type: ignore # lazy: only when an s3:// URI is actually used
30            from botocore.config import Config # type: ignore
31            self._client = boto3.client(
32                "s3",
33                endpoint_url=endpoint_url or os.environ.get("AWS_ENDPOINT_URL_S3") or None,
34                region_name=region,
35                config=Config(s3={"addressing_style": addressing_style}),
36            )
37
38            def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
39                if dst is None:
40                    raise ValueError("S3Storage.fetch_to_local requires a dst path")

```

```

41         dst = str(dst)
42         if os.path.exists(dst) and not overwrite:
43             return dst
44         os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
45         self._client.download_file(ref.bucket, ref.key, dst)
46         return dst
47
48     def put(self, local_path, ref, *, content_type=None) -> StorageRef:
49         extra = {"ContentType": content_type} if content_type else None
50         self._client.upload_file(str(local_path), ref.bucket, ref.key, ExtraArgs=extra)
51         return ref
52
53     def exists(self, ref) -> bool:
54         try:
55             self._client.head_object(Bucket=ref.bucket, Key=ref.key)
56             return True
57         except Exception: # noqa: BLE001 - any boto/client error means "not found /
unreachable"
58             return False
59
60     def stat(self, ref) -> StorageStat:
61         h = self._client.head_object(Bucket=ref.bucket, Key=ref.key)
62         return StorageStat(size=int(h.get("ContentLength", 0)),
modified=h.get("LastModified"),
63                             etag=h.get("ETag"), content_type=h.get("ContentType"))
64
65     def list(self, prefix) -> Iterator[StorageRef]:
66         paginator = self._client.get_paginator("list_objects_v2")
67         for page in paginator.paginate(Bucket=prefix.bucket, Prefix=prefix.key):
68             for obj in page.get("Contents", []) or []:
69                 key = obj["Key"]
70                 yield StorageRef("s3", prefix.bucket, key,
f"s3://{prefix.bucket}/{key}")
71
72     def delete(self, ref) -> None:
73         self._client.delete_object(Bucket=ref.bucket, Key=ref.key)
74
75     def signed_url(self, ref, *, ttl=3600, method="GET") -> str:
76         op = "get_object" if method.upper() == "GET" else "put_object"
77         return self._client.generate_presigned_url(
78             op, Params={"Bucket": ref.bucket, "Key": ref.key}, ExpiresIn=ttl or
self._ttl)
79

```

14. assistant (2026-06-20T16:56:35.154Z)

Now let me search for all write operations and understand the config structure:

15. user (2026-06-20T16:56:35.760Z)

```

backend\worker\__main__.py:98:     local_video = storage.fetch(args.video_uri,
os.path.join(scratch, "in.mp4"), config=storage_cfg)
backend\worker\__main__.py:153:     storage.put(os.path.join(out_local, fn), uri,
config=storage_cfg)
backend\worker\__main__.py:256:     label_uris = sorted(u for u in
storage.list_uris(f"{corpus}/labels/", config=storage_cfg)
backend\worker\__main__.py:271:     for vuri in storage.list_uris(f"{corpus}/videos/",
config=storage_cfg):
backend\worker\__main__.py:282:     local_label = storage.fetch(uri, os.path.join(labels_dir,
fname), config=storage_cfg)
backend\worker\__main__.py:288:     local_video = storage.fetch(vuri,
os.path.join(videos_dir, storage.parse_uri(vuri).name),
backend\worker\__main__.py:324:     floor_local = (storage.fetch(args.floor,
os.path.join(scratch, "floor.json"), config=storage_cfg)
backend\worker\__main__.py:339:     storage.put(os.path.join(bundle, fn), uri,

```

```
config=storage_cfg)
```

16. user (2026-06-20T16:56:36.249Z)

```
1      """Configuration package.
2
3      Public API:
4          from backend.config import load_config, AppConfig, Config
5
6      This package is the canonical place for all configuration access.
7      """
8
9      from .loader import load_config, save_default_config
10     from .models import ( # re-export common ones
11         AppConfig,
12         Config,
13         IndexingConfig,
14         StitchingConfig,
15         StorageConfig,
16         ValidationConfig,
17     )
18
19     __all__ = [
20         "load_config",
21         "save_default_config",
22         "AppConfig",
23         "Config",
24         "IndexingConfig",
25         "StitchingConfig",
26         "StorageConfig",
27         "ValidationConfig",
28     ]
29
```

17. user (2026-06-20T16:56:36.334Z)

```
1      """Configuration loader and saver.
2
3      Provides a single place to load validated AppConfig with built-in defaults
4      merged with optional file overrides (config.json).
5
6      This replaces scattered json.load + .get() patterns throughout the codebase.
7      """
8
9      from __future__ import annotations
10
11     import json
12     import logging
13     from pathlib import Path
14     from typing import Any
15
16     from pydantic import BaseModel
17
18     from .models import AppConfig
19
20     logger = logging.getLogger(__name__)
21
22     DEFAULT_CONFIG_PATH = Path("config.json")
23
24
25     def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str =
""") -> list[str]:
26         """Dotted paths in `data` that the typed config will not honor.
27
28         Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is
```

```

29     kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
30     typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
31     user's intent is lost ? collect every such key so load_config can warn.
32     """
33     unknown: list[str] = []
34     fields = model_cls.model_fields
35     for key, val in data.items():
36         if key not in fields:
37             unknown.append(prefix + key)
38             continue
39         ann = fields[key].annotation
40         if isinstance(val, dict) and isinstance(ann, type) and issubclass(ann,
BaseModel):
41             unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}."))
42     return unknown
43
44
45 def _get_builtin_defaults() -> dict[str, Any]:
46     """Return a plain dict of the current built-in defaults.
47
48     The AppConfig model is the single source of truth for defaults.
49     """
50     return AppConfig().model_dump(mode="json")
51
52
53 def load_config(path: str | Path | None = None) -> AppConfig:
54     """Load configuration with the following precedence:
55
56     1. Built-in defaults defined in the Pydantic models.
57     2. Values from the JSON file (if present and readable).
58     3. (Future) environment / CLI overrides.
59
60     Missing file or unreadable file ? returns a fully defaulted AppConfig.
61     Unknown keys in the file are tolerated (extra="allow" on the model).
62     """
63     cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
64
65     file_data: dict[str, Any] = {}
66     if cfg_path.exists():
67         try:
68             with open(cfg_path, "r", encoding="utf-8") as f:
69                 file_data = json.load(f)
70         except Exception as exc:
71             # Non-fatal: continue with defaults + whatever we could parse.
72             # In production we might log here.
73             logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
data.")
74
75     # A config.json that is valid JSON but not an OBJECT (e.g. `[ ]`, `42`, `"x"`,
`null`) would
76     # otherwise crash startup: a truthy non-mapping hits `.items()` in
_unknown_key_paths, and any
77     # non-mapping breaks the `**defaults, **file_data` unpack. Degrade to defaults +
warn, per the
78     # loader's "bad input is non-fatal" contract.
79     if not isinstance(file_data, dict):
80         logger.warning("%s is not a JSON object (got %s); ignoring it and using
defaults.",
81                        cfg_path, type(file_data).__name__)
82     file_data = {}
83
84     if file_data:
85         unknown = _unknown_key_paths(file_data, AppConfig)
86         if unknown:
87             logger.warning(

```

```

88         "[CONFIG WARNING] %s contains keys the typed config does not "
89         "recognize (typo'd or removed knobs ? top-level extras are kept "
90         "but unread; unknown section keys are silently dropped): %s",
91         cfg_path, ", ".join(sorted(unknown)),
92     )
93
94     # Merge: start from defaults, overlay file data (Pydantic will validate/coerce).
95     merged = {**_get_built_in_defaults(), **file_data}
96
97     return AppConfig.model_validate(merged)
98
99
100 def save_default_config(path: str | Path | None = None) -> Path:
101     """Write a fresh default config.json.
102
103     Used by scripts/doctor.py diagnostics and the --setup CLI path.
104     Overwrites if the file exists.
105     """
106     cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
107     cfg = AppConfig()
108     cfg_path.parent.mkdir(parents=True, exist_ok=True)
109     cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
110     return cfg_path
111
112
113 # Backwards-compatible alias used by some older call sites during transition.
114 def get_default_config() -> dict[str, Any]:
115     """Return defaults as a plain dict (for transitional use only)."""
116     return _get_built_in_defaults()
117

```

18. assistant (2026-06-20T16:56:38.306Z)

Now let me check the models to understand storage config:

19. user (2026-06-20T16:56:38.835Z)

find: 'C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config': No such file or directory

20. user (2026-06-20T16:56:40.769Z)

```

backend\config\loader.py
backend\config\__init__.py
backend\config\models.py

```

21. user (2026-06-20T16:56:44.347Z)

```

1     """Pydantic models for the application configuration.
2
3     This module defines the canonical, validated, typed configuration for the
4     Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6     The models support graceful migration from old config.json files via extra="allow".
7     """
8
9     from __future__ import annotations
10
11     from typing import Any, List, Literal, Optional
12
13     from pydantic import BaseModel, Field, field_validator, model_validator
14
15     Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17

```

```

18     class ValidationRelevanceConfig(BaseModel):
19         court_green_lower: list[int] = Field(default=[35, 40, 40])
20         court_green_upper: list[int] = Field(default=[85, 255, 255])
21         court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24     class ValidationConfig(BaseModel):
25         min_resolution_width: int = 1280
26         min_resolution_height: int = 720
27         min_fps: float = 29.9
28         min_blur_threshold: float = 20.0
29         max_scene_cuts_per_minute: float = 0.5
30         relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)
31
32
33     class TrackNetConfig(BaseModel):
34         wsl_distro: str = "Ubuntu"
35         repo_dir: str = "~/models/TrackNetV4"
36         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37         conda_env: str = "TrackNetV4"
38         weights_path: str = ""
39         queue_length: int = 5
40         stage_in_wsl: bool = True
41         wsl_stage_dir: str = "~/clips"
42         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43         velocity_threshold: float = 0.02
44         # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45         # decoded frame cache) are deleted automatically ? they are pure intermediates,
46         # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47         # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48         keep_frames: bool = False
49         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50         # 0 = disabled.
51         min_free_gb: float = 0.0
52
53
54     class WasbIndexConfig(BaseModel):
55         """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57         Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58         these knobs actually take effect ? previously this whole block was silently dropped
59         because nested config sections defaulted to extra='ignore'.
60         """
61         wsl_distro: str = "Ubuntu"
62         repo_dir: str = "~/models/WASB-SBDT"
63         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64         conda_env: str = "wasb"
65         weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66         sport: str = "badminton"
67         wsl_stage_dir: str = "~/clips"
68         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69         velocity_threshold: float = 0.02
70         # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
71         ---
72         # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
73         device: str = "cuda" # torch device for the native runner: cuda | mps | cpu
74         python_bin: str = "python" # the `wasb` conda env python (invoked by path, no
`conda activate`)
75         # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
76         # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
77         # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.

```

```

77         keep_frames: bool = False
78         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
79         min_free_gb: float = 0.0
80         # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
81         # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
82         # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
83         # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
84         # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).
85         # Tri-state:
86         #     None          = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
87         #                   behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
88         #     True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
89         stream_video: Optional[bool] = None
90
91
92     class FusionConfig(BaseModel):
93         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
94
95         Every default reproduces control behaviour: the fusion segmenter persists the
96         shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
97         true ? the person-cue features, but it does NOT gate the served handoff unless a
98         threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
99         is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
100        supplied ? off-court persons (spectators / adjacent courts) are excluded.
101        """
102        # Which trajectory substrate seeds the windows (shuttle stays primary).
103        substrate: Literal["wasb", "tracknet"] = "wasb"
104        # Windowing velocity threshold for the fusion family (the engine reads
105        # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
106        # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
107        velocity_threshold: float = 0.02
108        # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
109        # enabled ? so the default path never imports torch / touches the GPU.
110        cue_flags: dict[str, bool] = Field(default_factory=dict)
111        # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
112        # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
113        min_crossings: int = Field(default=0, ge=0)
114        # Served Gate B: when the person cue is on, drop windows with median in-court
players
115        # < this. 0 = OFF.
116        min_players: int = Field(default=0, ge=0)
117        # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
118        court_polygon: Optional[list[list[float]]] = None
119        # Net line for the person two-sided-motion split (person features only ? the shuttle
120        # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
121        net_axis: Literal["x", "y"] = "y"
122        net_position: float = Field(default=0.5, ge=0.0, le=1.0)
123
124
125     class IndexingConfig(BaseModel):
126         default_segmenter: str = "yolo_hybrid"

```

```

127     use_gpu: bool = True
128     # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
129     # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
130     # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
131     # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
132     # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
133     detector_impl: str = "auto"
134     motion_threshold: float = 0.08
135     min_segment_duration: float = 3.0
136     dead_time_merge_gap: float = 2.0
137     chunk_duration: float = 90.0
138     chunk_overlap: float = 10.0
139     detector_model: str = "fasterrcnn_resnet50_fpn"
140     detector_score_threshold: float = 0.5
141     yolo_velocity_threshold: float = 0.15
142     yolo_frame_skip: int = 3
143     yolo_tracker: str = "bytetrack.yaml"
144     yolo_prefetch_workers: int = 2
145     min_action_window_duration: float = 2.0
146     # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
147     # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
148     # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
149     # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
150     # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
151     # golden, R2 toIF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
152     # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
153     max_window_duration: float = 6.0
154     window_min_split_gap: float = 0.5 # min internal gap (s) that qualifies as a split
point
155     # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
156     # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
157     # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
158     # Only cuts (recall can't drop). OFF by default until measured/promoted.
159     window_hard_cap: bool = False
160     # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
161     # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
162     # [?end] gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
163     # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
164     # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
165     # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, [?end] 4.96?3.31s (n=13).
The W1 cap
166     # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
167     window_inactivity_end: float = 1.5
168     inactivity_rally_thresh: float = 0.20
169     inactivity_min_density: float = 0.30
170     # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
171     # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group

```

```

STILL longer
172         # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
173         # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
174         # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
175         # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
176         # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
177         # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
178         window_blob_fallback: bool = False
179         # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
180         # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
181         # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
182         # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
183         window_blob_factor: float = 4.0
184         log_yolo_candidates: bool = True
185         store_yolo_candidates: bool = True
186         ai_handoff_padding: float = 2.0
187         ai_max_workers: int = 5
188         # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
189         # chunks of a high-bitrate (4K) source blow past the provider upload cap
190         # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
191         # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
192         # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
193         # Matches gemini_refine's --clip-scale-width default.
194         ai_chunk_scale_width: int = 1280
195         # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
196         # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
197         # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
198         # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
199         # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
200         skip_ai_handoff: bool = False
201         # Optional debug knob: trim every video to the first N seconds before processing.
202         debug_duration: Optional[float] = None
203
204         tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
205         wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
206         fusion: FusionConfig = Field(default_factory=FusionConfig)
207
208         @field_validator("default_segmenter")
209         @classmethod
210         def segmenter_must_be_registered(cls, v: str) -> str:
211             # Registry check happens at runtime in main/segmenter selection.
212             # Here we just allow any string for forward compatibility.
213             return v
214
215         @model_validator(mode="after")
216         def _chunk_step_must_be_positive(self) -> "IndexingConfig":
217             # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
218             # step makes `while t < duration: t += step` loop forever, growing memory before
any
219             # GPU work. Reject the bad config at load time instead.
220             if self.chunk_overlap >= self.chunk_duration:

```

```

221         raise ValueError(
222             f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
223         "
224             f"({self.chunk_duration}); otherwise chunking never advances."
225         )
226     return self
227
228 class OutputConfig(BaseModel):
229     default_highlights_name: str = "highlights.mp4"
230     db_name: str = "sports_indexer.db"
231     # Directory where the DB, highlight reels, and processed clips are written.
232     # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
233     output_dir: str = "output"
234
235
236 class WatermarkConfig(BaseModel):
237     """Branding overlay burned into each highlight clip during the per-clip re-encode
238     (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
239     (under `stitching.watermark` in config.json) to turn it off. The text is fully
240     configurable. The concat stage stays stream-copy (-c copy)."""
241     enabled: bool = True
242     text: str = "Generated by Khelsutra.guru"
243     logo_path: str = "" # optional transparent PNG overlay; "" = no logo
244     font_path: str = "" # optional .ttf; "" = use the fontconfig `font` family
245     below
246     font: str = "Sans" # fontconfig family used when font_path is empty
247     font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5) # text height as a
248     fraction of frame height
249     opacity: float = Field(default=0.85, ge=0.0, le=1.0)
250     box: bool = True # faint backing box behind the text for legibility
251     margin: int = Field(default=24, ge=0) # px inset from the chosen corner
252     position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
253     right"
254
255 class StitchingConfig(BaseModel):
256     begin_padding: float = 0.5
257     end_padding: float = 0.5
258     use_cache: bool = False
259     min_shot_count: int = 1
260     # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
261     duration
262     # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
263     # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
264     the
265     # fully-local no-AI path, whereas duration is derived from the rally's own
266     start/end. The
267     # local detector over-fires and its false positives are mostly SHORT (motion blips,
268     # background-court fragments), so this keeps the reel to the eye-catching long
269     rallies.
270     # OFF by default ? the detector output is unchanged, only which rallies reach the
271     reel.
272     min_rally_duration: float = Field(default=0.0, ge=0.0)
273     watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
274
275
276 class LoggingConfig(BaseModel):
277     """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
278     # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
279     # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
280     # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
281     # them to WARNING; set true to restore full chatter for request-flow debugging.
282     verbose_http: bool = False

```

```

277
278 class SecurityConfig(BaseModel):
279     # When set, /api/process video_path must resolve under this directory (hosted/tenant
280     # mode). Default None preserves the single-user local 'any local file' workflow.
281     allowed_video_root: Optional[str] = None
282
283     # --- Chunked upload (B2, PR-2) ---
284     # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
285     # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
286     # contain upload_dir or /api/process will reject the uploaded paths.
287     upload_dir: str = "output/uploads"
288     # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
289     # free-tier edge proxies cap request bodies there (HOSTING_PLAN $2).
290     max_upload_mb: int = 4096
291     max_part_mb: int = 95
292     allowed_upload_extensions: List[str] = ["mp4", "mov"]
293
294
295 class StorageConfig(BaseModel):
296     """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
297     behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
298     Spaces
299     + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
300     locally MOUNTED
301     Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
302     the constructor ?
303     **secrets are never stored here**, only endpoints / regions / file paths / source-
304     selectors
305     (boto3/google auth chains supply credentials). See ``backend/storage/``. """
306     backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
307     # Output root for produced artifacts. "" => fall back to output.output_dir (non-
308     breaking).
309     output_root: str = ""
310     local: dict[str, Any] = Field(default_factory=dict)
311     s3: dict[str, Any] = Field(default_factory=dict)
312     gcs: dict[str, Any] = Field(default_factory=dict)
313     gdrive: dict[str, Any] = Field(default_factory=dict)
314
315
316 class AppConfig(BaseModel):
317     """Root configuration object.
318
319     All runtime configuration should be accessed via an instance of this class
320     (or the loader) rather than raw dict lookups.
321
322     """
323     sport: Sport = "badminton"
324     ai_provider: str = "gemini"
325     # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
326     # flagship;
327     # the alias never deprecates underneath us. Eval reports record the run date
328     # alongside.
329     ai_model: str = "gemini-flash-latest"
330
331     validation: ValidationConfig = Field(default_factory=ValidationConfig)
332     indexing: IndexingConfig = Field(default_factory=IndexingConfig)
333     output: OutputConfig = Field(default_factory=OutputConfig)
334     stitching: StitchingConfig = Field(default_factory=StitchingConfig)
335     security: SecurityConfig = Field(default_factory=SecurityConfig)
336     logging: LoggingConfig = Field(default_factory=LoggingConfig)
337     storage: StorageConfig = Field(default_factory=StorageConfig)
338
339     # Allow extra keys during transition so existing config.json files do not break.
340     model_config = {
341         "extra": "allow",

```

```

335         "validate_assignment": True,
336     }
337
338     # Temporary migration shim for legacy dict-style access (config.get("section",
339     {}).get(...)).
340     # Will be removed once all call sites (validators, segmenters, etc.) move to typed
341     # attribute access / the typed config. A nested *section* is returned as a plain
342     dict
343     # via model_dump, so legacy chained access keeps working ? returning the Pydantic
344     # section model instead breaks it (a model has no .get), which bit every validator
345     on
346     # real runs. Scalar leaves are returned as-is.
347     def get(self, key: str, default: Any = None) -> Any:
348         _MISSING = object()
349         val: Any = _MISSING
350         if key in type(self).model_fields or hasattr(self, key):
351             val = getattr(self, key, _MISSING)
352         if val is _MISSING:
353             # Legacy convenience: find a leaf key inside any nested section. Sections
354             # are
355             # discovered dynamically from the model fields, so a future top-level
356             # section
357             # (e.g. `audio`, `jobs`) is covered automatically without editing a
358             # hardcoded list.
359             for fname in type(self).model_fields:
360                 section = getattr(self, fname, None)
361                 if isinstance(section, BaseModel) and hasattr(section, key):
362                     val = getattr(section, key)
363                     break
364             if val is _MISSING:
365                 return default
366             if isinstance(val, BaseModel):
367                 return val.model_dump(mode="json")
368             return val
369
370     def __getitem__(self, key: str) -> Any:
371         val = self.get(key)
372         if val is None and key not in type(self).model_fields:
373             # Re-raise like a real dict for unknown top-level
374             raise KeyError(key)
375         return val
376
377     # Convenience type alias for callers that want to annotate
378     Config = AppConfig
379
380

```

22. assistant (2026-06-20T16:56:46.396Z)

Now let me search for more write operations in the codebase:

23. user (2026-06-20T16:56:47.273Z)

```

run_regression.py:29:     return os.path.join(cfg.output.output_dir, cfg.output.db_name)
run_eval.py:33:     """Resolve the DB path from the typed config (output.output_dir /
run_eval.py:35:     return os.path.join(cfg.output.output_dir, cfg.output.db_name)
artifacts\so.py:7:[Omitted long matching line]
scripts\run_process_and_stitch.py:30:                                     help="Where to write the stitched
reel (default: config output.output_dir).")
scripts\run_process_and_stitch.py:40:     output_dir = args.output_dir or cfg.output.output_dir
# no hardcoded OS path
scripts\run_process_and_stitch.py:47:     os.makedirs(cfg.output.output_dir, exist_ok=True)
scripts\run_process_and_stitch.py:48:     db_path = os.path.join(cfg.output.output_dir,
cfg.output.db_name)

```

```

scripts\run_process_and_stitch.py:158:    compiler = VideoCompiler(output_dir,
end_padding=end_padding, begin_padding=begin_padding,
scripts\nightly_regression.py:71:DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output",
"nightly_regression")
scripts\nightly_regression.py:72:DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "output",
"human_lovo_manifest.json")
scripts\nightly_regression.py:73:DEFAULT_FEATURES_DIR = os.path.join(REPO_ROOT, "output")
scripts\nightly_regression.py:540:    os.makedirs(os.path.dirname(path), exist_ok=True)
scripts\nightly_regression.py:608:    os.makedirs(args.report_dir, exist_ok=True)
scripts\run_phase3_eval.py:92:    db_path = args.db or os.path.join(cfg.output.output_dir,
cfg.output.db_name)
scripts\run_phase3_eval.py:93:    os.makedirs(os.path.dirname(db_path) or ".", exist_ok=True)
tests\test_api_endpoints.py:104:# Item 4 positive tests (output layout honors configured
output_dir)
tests\test_api_endpoints.py:107:def
test_process_uses_configured_output_dir_not_literal_output(env, tmp_path, monkeypatch):
tests\test_api_endpoints.py:108:    """Positive test for Item 4: when output.output_dir
overridden, artifacts/paths
tests\test_api_endpoints.py:111:    This would have caught re-introduction of hard-coded
os.path.join("output", ...)
tests\test_api_endpoints.py:121:    cfg.output.output_dir = custom_out
tests\test_api_endpoints.py:125:    os.makedirs(custom_out, exist_ok=True)
tests\test_api_endpoints.py:138:    assert cfg.output.output_dir == custom_out
tests\test_api_endpoints.py:140:    literal = os.path.join("output", vid)
tests\test_api_endpoints.py:145:def
test_compiler_and_reingest_safety_honor_output_dir_override(env, tmp_path, monkeypatch):
tests\test_api_endpoints.py:147:    Override output_dir; failed run must not pollute literal
'output/'; success uses configured.
tests\test_api_endpoints.py:156:    cfg.output.output_dir = custom_out
tests\test_api_endpoints.py:158:    os.makedirs(custom_out, exist_ok=True)
tests\test_api_endpoints.py:167:    assert cfg.output.output_dir == custom_out
tests\test_api_endpoints.py:169:    literal = os.path.join("output", vid)
tests\test_api_endpoints.py:176:    assert cfg.output.output_dir == custom_out # still honored
tests\test_api_endpoints.py:178:    literal = os.path.join("output", vid)
tests\test_api_endpoints.py:269:    cfg.output.output_dir = str(tmp_path / "out")
training\gen0\harness.py:156:    os.makedirs(bundle_dir, exist_ok=True)
tests\test_ablation.py:497:    os.makedirs(d)
tests\test_ablation.py:515:    os.makedirs(d)
tests\test_ablation.py:526:    os.makedirs(d)
tests\test_config.py:226:def test_output_dir_field_default_and_override():
tests\test_config.py:227:    """output_dir now lives on OutputConfig (was a write-only runtime
hack), so
tests\test_config.py:231:    assert cfg.output.output_dir == "output" # built-in default
tests\test_config.py:233:    # The CLI override path: main() assigns args.output_dir onto the
typed model.
tests\test_config.py:234:    cfg.output.output_dir = "/tmp/custom_out"
tests\test_config.py:235:    assert cfg.output.output_dir == "/tmp/custom_out"
tests\test_config.py:241:    json.dump({"output": {"output_dir": "reels", "db_name":
"x.db"}}, f)
tests\test_config.py:243:    assert loaded.output.output_dir == "reels"
tools\generate_highlights.py:71:    os.makedirs(os.path.dirname(dest_path), exist_ok=True)
tools\generate_highlights.py:98:    os.makedirs(out_dir, exist_ok=True)
tools\generate_highlights.py:99:    os.makedirs(local_work_dir, exist_ok=True)
tools\generate_highlights.py:291:    ap.add_argument("--local-work",
default=os.path.join("output", "highlights_work"),
tools\generate_highlights.py:293:    ap.add_argument("--db", default=os.path.join("output",
"sports_indexer.db"))
tools\cleanup_caches.py:171:def scan_windows(output_dir: str) -> List[Entry]:
tools\cleanup_caches.py:174:    for base in (output_dir, os.path.join(output_dir, "wasb")):
tools\cleanup_caches.py:179:        if base == output_dir and de.name == "wasb":
tools\cleanup_caches.py:244:    entries = scan_wsl(args.distro, args.stage_dir) +
scan_windows(args.output_dir)
backend\api\uploads.py:79:    os.makedirs(upload_dir, exist_ok=True)
backend\api\uploads.py:89:@router.put("/api/upload/{upload_id}/part/{index}")
backend\config\models.py:233:    output_dir: str = "output"

```

```

backend\config\models.py:303:    # Output root for produced artifacts. "" => fall back to
output.output_dir (non-breaking).
docs\archives\quality-iterations\2026-06-18-lovo-resweep-n13\lovo_resweep.py:26:MANIFEST =
os.path.join(MAIN, "output", "human_lovo_manifest.json")
docs\archives\quality-iterations\2026-06-18-lovo-resweep-n13\lovo_resweep.py:27:FEATURES =
os.path.join(MAIN, "output")
backend\worker\_main_.py:94:    os.makedirs(scratch, exist_ok=True)
backend\worker\_main_.py:95:    config.output.output_dir = scratch
backend\worker\_main_.py:107:    db = Database(os.path.join(scratch, config.output.db_name))
backend\worker\_main_.py:144:    out_local = os.path.join(scratch, "outputs", video_id)
backend\worker\_main_.py:145:    os.makedirs(out_local, exist_ok=True)
backend\worker\_main_.py:153:    storage.put(os.path.join(out_local, fn), uri,
config=storage_cfg)
backend\worker\_main_.py:252:    os.makedirs(labels_dir, exist_ok=True)
backend\worker\_main_.py:253:    os.makedirs(traj_dir, exist_ok=True)
backend\worker\_main_.py:267:    os.makedirs(videos_dir, exist_ok=True)
backend\worker\_main_.py:339:    storage.put(os.path.join(bundle, fn), uri,
config=storage_cfg)
backend\reporting\_init_.py:72:    output_dir = (config.get("report") or
{}).get("output_dir")
backend\reporting\_init_.py:73:    res = rec.write(output_dir=output_dir)
backend\storage\gcs.py:37:    os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
backend\storage\local.py:38:    os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
backend\storage\local.py:45:    os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
backend\main.py:49:os.makedirs("backend/api/static", exist_ok=True)
backend\main.py:229:    # report.output_dir). Never raises. Surface the paths in the
response.
backend\main.py:310:    (which only writes into output_dir; the browser previously got back an
unreachable
backend\main.py:318:    output_dir = getattr(getattr(global_config, "output", None),
"output_dir", "output") or "output"
backend\main.py:319:    path = os.path.join(output_dir, name)
backend\main.py:321:    path = resolve_under_root(path, output_dir)
backend\main.py:389:    output_dir = getattr(getattr(global_config, "output", None),
"output_dir", "output") or "output"
backend\main.py:390:    compiler = VideoCompiler(output_dir,
begin_padding=stitching.begin_padding,
backend\main.py:514:    out_dir = args.output_dir
backend\main.py:518:    os.makedirs(out_dir, exist_ok=True)
backend\main.py:546:    # on the typed model (OutputConfig.output_dir), so every consumer ?
including
backend\main.py:548:    global_config.output.output_dir = args.output_dir
backend\main.py:549:    os.makedirs(global_config.output.output_dir, exist_ok=True)
backend\main.py:551:    db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
backend\storage\gdrive.py:119:    self._local.put(local_path, self._local_ref(ref),
content_type=content_type)
tests\test_fusion_compare.py:137:    os.makedirs(out_dir)
tests\test_fusion_compare.py:169:    os.makedirs(out_dir)
backend\storage\_init_.py:68:    get_storage(ref.backend, config=config).put(local_path, ref,
content_type=content_type)
backend\eval\export_wasb_segments.py:67:    os.makedirs(osp.dirname(osp.abspath(path)) or ".",
exist_ok=True)
backend\eval\experiment.py:616:    os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
tests\test_fusion_golden_telemetry.py:65:    os.makedirs(out_dir)
tests\test_fusion_golden_telemetry.py:103:    os.makedirs(out_dir)
tests\test_fusion_golden_telemetry.py:132:    os.makedirs(out_dir)
tests\test_fusion_golden_telemetry.py:160:    os.makedirs(out_dir)
backend\utils\paths.py:9:    `os.path.join(output_dir, name)` silently discards `output_dir`
when `name` is absolute,

```

[Showing results with pagination = limit: 100]

Let me look at the reporting module and training harness:

25. user (2026-06-20T16:56:49.964Z)

```
1      """Per-video observability reports for the indexer.
2
3      `obsreport` is a SOFT dependency: if it is not installed, this module degrades to a
4      no-op and the pipeline runs exactly as before. The single entrypoint
5      `emit_indexer_report(...)` is designed to be called from a `finally` block and
6      NEVER raises ? emitting a report must not be able to break a processing run.
7      """
8      from __future__ import annotations
9
10     from typing import Any, Dict, List, Optional
11
12     from . import run_signals # no obsreport dependency ? always importable
13
14     try: # soft dependency ? never hard-fail the pipeline on a reporting import
15         import obsreport # noqa: F401
16         from . import harvest
17
18         OBSREPORT_AVAILABLE = True
19     except Exception: # pragma: no cover - exercised only when obsreport is absent
20         OBSREPORT_AVAILABLE = False
21
22
23     def report_enabled(config: Optional[Dict[str, Any]]) -> bool:
24         return bool((config or {}).get("report", {}).get("enabled", True))
25
26
27     def emit_indexer_report(
28         *,
29         db,
30         video_id: str,
31         video_path: Optional[str],
32         config: Optional[Dict[str, Any]],
33         val_results: List[Any],
34         val_context: Optional[Dict[str, Any]] = None,
35         segmenter_requested: Optional[str] = None,
36         segmenter_actual: Optional[str] = None,
37         was_reingest: bool = False,
38         segmentation_ran: bool = True,
39     ) -> Optional[Dict[str, str]]:
40         """Harvest + write the report. Returns the written file paths, or None if
41         reporting is unavailable/disabled/failed. Never raises."""
42         config = config or {}
43         # Accept either a plain dict or a typed config object (e.g.
44         backend.config.AppConfig).
45         # Normalize to a plain nested dict so harvest's dict access works uniformly.
46         if hasattr(config, "model_dump"):
47             try:
48                 config = config.model_dump(mode="json")
49             except Exception:
50                 config = {}
51         # Consume run signals unconditionally (even when reporting is off) so the
52         # process-local store never accumulates entries for finished runs.
53         signals = run_signals.pop(video_id)
54         if not OBSREPORT_AVAILABLE or not report_enabled(config):
55             return None
56         try:
57             play_segments = db.query_play_segments(video_id, {}) if db is not None else []
58             candidates = db.query_candidate_segments(video_id) if db is not None else []
59             rec = harvest.record_run(
60                 video_id=video_id,
61                 video_path=video_path,
```

```

61         config=config,
62         val_results=val_results,
63         val_context=val_context,
64         play_segments=play_segments,
65         candidates=candidates,
66         segmenter_requested=segmenter_requested,
67         segmenter_actual=segmenter_actual,
68         was_reingest=was_reingest,
69         segmentation_ran=segmentation_ran,
70         run_signals=signals,
71     )
72     output_dir = (config.get("report") or {}).get("output_dir")
73     res = rec.write(output_dir=output_dir)
74     return res.paths or None
75 except Exception: # pragma: no cover - defensive; reporting must never break a run
76     return None
77

```

26. user (2026-06-20T16:56:50.165Z)

find: 'C:\Users\avidu\Projects\badminton-highlight-indexer\training': No such file or directory

27. user (2026-06-20T16:56:52.228Z)

```

training\gen0\__init__.py
training\__init__.py
.claude\worktrees\bhi-execpolicy\training\gen0\__init__.py
.claude\worktrees\bhi-execpolicy\training\__init__.py
.claude\worktrees\bhi-execpolicy\training\gen0\harness.py
training\gen0\harness.py
.claude\worktrees\agent-a771ff1d85e18f2ed\training\__init__.py
.claude\worktrees\agent-a771ff1d85e18f2ed\training\gen0\__init__.py
.claude\worktrees\agent-a771ff1d85e18f2ed\training\gen0\harness.py
.claude\worktrees\tier1-windowing\training\gen0\harness.py
.claude\worktrees\tier1-windowing\training\gen0\__init__.py
.claude\worktrees\tier1-windowing\training\__init__.py
.claude\worktrees\w2-safeguard\training\gen0\harness.py
.claude\worktrees\w2-safeguard\training\gen0\__init__.py
.claude\worktrees\w2-safeguard\training\__init__.py
.claude\worktrees\hardchop\training\__init__.py
.claude\worktrees\hardchop\training\gen0\__init__.py
.claude\worktrees\hardchop\training\gen0\harness.py
.claude\worktrees\boxhill17\training\gen0\harness.py
.claude\worktrees\boxhill17\training\gen0\__init__.py
.claude\worktrees\boxhill17\training\__init__.py
.claude\worktrees\pr-stitch\training\__init__.py
.claude\worktrees\pr-stitch\training\gen0\__init__.py
.claude\worktrees\pr-stitch\training\gen0\harness.py
.claude\worktrees\gap-doc\training\__init__.py
.claude\worktrees\gap-doc\training\gen0\harness.py
.claude\worktrees\gap-doc\training\gen0\__init__.py
.claude\worktrees\resweep\training\__init__.py
.claude\worktrees\resweep\training\gen0\harness.py
.claude\worktrees\resweep\training\gen0\__init__.py
.claude\worktrees\recdoc\training\__init__.py
.claude\worktrees\recdoc\training\gen0\harness.py
.claude\worktrees\recdoc\training\gen0\__init__.py

```

28. user (2026-06-20T16:56:53.986Z)

```

1     """Gen-0 TAL harness (M0.4): ONE command -> train -> honest LOVO eval -> ship-gate ->
artifact.
2
3     python -m training.gen0.harness --manifest output/human_lovo_manifest.json \

```

```

4         --floor eval_baselines/heuristic_lovo_n6.json --out artifacts/rally_tal_gen0
--version 0.1.0
5
6     Productionizes the rally_seq prototype per ADR-008:
7     - TRAINS on the corpus manifest (per-video trajectory + golden labels), evaluates with
8       honest leave-one-video-out (split by match, threshold tuned on train only), then fits
9       the FINAL model on all videos.
10    - Emits a versioned ARTIFACT BUNDLE ? the only thing serving may ever consume:
11        <out>/<version>/weights.json    (gitignored; classifier + frozen normalization)
12        <out>/<version>/manifest.json    (the reviewable contract: feature-schema version,
13                                          calibration, training lineage + GT hashes,
14                                          eval results, attestations, gate verdict)
15    - Runs the SHIP-GATE as pre-flight: floor comparison + a PAIRED per-video sign test
16      (at n=6 a mean gap alone is noise-ambiguous), plus the hard attestation blocks
17      (WASB C3 unresolved => commercial ship refused; explicit F1@tIoU target still
18      undefined => no self-declared victory). The product side owns final admission
19      (backend/eval/regression.py); this harness never grades itself into serving.
20
21    Training may import backend.* (one-way wall); backend must never import training.*.
22    """
23    from __future__ import annotations
24
25    import argparse
26    import hashlib
27    import json
28    import os
29    import subprocess
30    import time
31    from math import comb
32    from typing import Dict, List, Optional
33
34    import numpy as np
35
36    from backend.eval.classifier import LogisticRegression
37    from backend.eval.gt_loader import load_gt_intervals
38    from backend.eval.regression import gt_hash
39    from backend.eval.rally_seq_proto import (
40        labels_for, run as lovo_run, _seg_f1,
41    )
42    from backend.features.rally_seq import FEAT_NAMES, FEATURE_SCHEMA, build_frame_arrays,
features
43    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
44
45    # Post-processing constants ? part of the model's identity, recorded in the manifest.
46    CALIBRATION_DEFAULTS = {"smooth": 7, "merge_gap": 1.0, "min_dur": 1.0}
47    THRESHOLD_GRID = [round(t, 2) for t in np.arange(0.2, 0.81, 0.05)]
48
49
50    def _git_sha() -> str:
51        try:
52            return subprocess.run(["git", "rev-parse", "HEAD"], capture_output=True,
53                                text=True, timeout=10).stdout.strip() or "unknown"
54        except Exception:
55            return "unknown"
56
57
58    def _sha256(path: str) -> str:
59        h = hashlib.sha256()
60        with open(path, "rb") as f:
61            for chunk in iter(lambda: f.read(65536), b''):
62                h.update(chunk)
63        return h.hexdigest()
64
65
66    def sign_test(learned: List[float], floor: List[float]):

```

```

67     """Paired one-sided sign test: is learned > floor more often than chance?
68
69     At n=6 a mean comparison alone is inside noise (SE ~0.11); the paired per-video
70     wins are the defensible statistic. Returns (wins, n, p_one_sided)."""
71     pairs = [(lv, fv) for lv, fv in zip(learned, floor) if lv != fv] # ties drop,
standard
72     wins = sum(1 for lv, fv in pairs if lv > fv)
73     n = len(pairs)
74     p = sum(comb(n, k) for k in range(wins, n + 1)) / (2 ** n) if n else 1.0
75     return wins, n, p
76
77
78     def train_final(specs: List[Dict]):
79         """Fit the FINAL model on ALL corpus videos; threshold tuned on the same (train-
side).
80
81         The honest generalization estimate is the LOVO result, not this fit ? recorded as
such."""
82         vids = []
83         for s in specs:
84             pts = TrackNetRunner.parse_trajectory_csv(s["trajectory"])
85             arr = build_frame_arrays(pts, float(s["frame_width"]), float(s["fps"]))
86             X, times = features(arr, float(s["fps"]))
87             gts = load_gt_intervals(s["labels"], strict=False)
88             vids.append({"name": s["name"], "X": X, "times": times,
89                         "y": labels_for(times, gts), "gts": gts})
90         Xall = np.vstack([v["X"] for v in vids])
91         yall = np.concatenate([v["y"] for v in vids])
92         clf = LogisticRegression(lr=0.2, epochs=2000, l2=1e-2)
93         clf.fit(Xall, yall, feature_names=FEAT_NAMES)
94         thr = max(THRESHOLD_GRID, key=lambda t: _seg_f1(clf, vids, float(t)))
95         return clf, float(thr)
96
97
98     def gate_verdict(eval_res: dict, floor: Optional[dict]) -> dict:
99         """Pre-flight ship-gate.
100
101         Provisional F1 target (NOT owner-ratified; calibration tracked in #172).
102         Floor (necessary): heuristic LOVO ~0.480 (from quality table).
103         Multi-court ref (makes owned \"worth serving\" per data): 0.66.
104         Proposed for \"very high P/R\" + significance: F1@tIoU0.5 >= 0.70 on
105         multi-court folds with paired-sign p<0.05 vs heuristic (refine as corpus grows).
106         Beating floor + sign test necessary; C3 main commercial blocker.
107         ship=True only when all non-C3 blocks clear (honesty first).
108         """
109         reasons: List[str] = []
110         floor_passed = None
111         sign = None
112         if floor:
113             floor_mean = float(floor["mean_f1"])
114             floor_passed = eval_res["mean_f1"] > floor_mean
115             if not floor_passed:
116                 reasons.append(f"LOVO mean {eval_res['mean_f1']:.3f} <= floor
{floor_mean:.3f}")
117             per = floor.get("per_video", {})
118             paired = [(r["f1"], per[r["video"]]) for r in eval_res["per_video"] if
r["video"] in per]
119             if paired:
120                 wins, n, p = sign_test([lv for lv, _ in paired], [fv for _, fv in paired])
121                 sign = {"wins": wins, "n": n, "p_one_sided": round(p, 4)}
122                 if p >= 0.05:
123                     reasons.append(f"paired sign test not significant ({wins}/{n} wins,
p={p:.3f}) "
124                                   f"? grow the corpus before claiming superiority")
125             else:

```

```

126         reasons.append("no floor baseline provided ? nothing to beat")
127
128     # Provisional blocker (per owner review on #170; see #172 for calibration).
129     # (The number itself is not the blocker; calibration tracked in #172.)
130     reasons.append("ship-gate F1@tIoU target NOT YET CALIBRATED ? provisional; see issue
#172")
131     reasons.append("WASB checkpoint provenance C3 UNRESOLVED ? commercial ship blocked
regardless of F1")
132     # ship remains False until C3 resolved + other gates (by design).
133     return {"ship": False, "floor_passed": floor_passed, "sign_test": sign, "blockers":
reasons}
134
135
136     def run(manifest_path: str, out_dir: str, version: str,
137            floor_path: Optional[str] = None) -> dict:
138         with open(manifest_path, encoding="utf-8") as f:
139             specs = json.load(f)
140             floor = None
141             if floor_path:
142                 with open(floor_path, encoding="utf-8") as f:
143                     floor = json.load(f)
144
145             # 1) Honest LOVO eval (the number that matters).
146             eval_res = lovo_run(specs)
147
148             # 2) Final model on all videos.
149             clf, thr = train_final(specs)
150
151             # 3) Gate (pre-flight).
152             verdict = gate_verdict(eval_res, floor)
153
154             # 4) Artifact bundle.
155             bundle_dir = os.path.join(out_dir, version)
156             os.makedirs(bundle_dir, exist_ok=True)
157             weights_path = os.path.join(bundle_dir, "weights.json")
158             payload = clf.to_dict()
159             payload["calibration"] = {"threshold": thr, **CALIBRATION_DEFAULTS}
160             payload["feature_schema_version"] = FEATURE_SCHEMA["version"]
161             with open(weights_path, "w", encoding="utf-8") as f:
162                 json.dump(payload, f, indent=2)
163
164             manifest = {
165                 "artifact": {
166                     "name": "rally_tal_gen0", "version": version,
167                     "created_at": time.strftime("%Y-%m-%dT%H:%M:%S%z"),
168                     "git_sha": _git_sha(),
169                     "weights_file": "weights.json", "weights_sha256": _sha256(weights_path),
170                 },
171                 "feature_schema": FEATURE_SCHEMA,
172                 "calibration": {"threshold": thr, **CALIBRATION_DEFAULTS,
173                                "note": "threshold tuned train-side on the full corpus; "
174                                "honest generalization = the LOVO eval below"},
175                 "training_lineage": {
176                     "corpus_manifest": manifest_path,
177                     "split_protocol": "leave-one-video-out by match",
178                     "label_source": "human (rally-annotator golden CSVs)",
179                     "videos": [{ "name": s["name"], "labels": s["labels"],
180                                "gt_hash": gt_hash(load_gt_intervals(s["labels"],
strict=False)),
181                                "fps": s["fps"]} for s in specs],
182                 },
183                 "eval": {
184                     "protocol": "LOVO, F1@tIoU=0.5 segment-level + per-frame AUC",
185                     "per_video": eval_res["per_video"],
186                     "mean_f1": eval_res["mean_f1"],

```

```

187         "floor": floor,
188     },
189     "attestation": {
190         "deps_licenses": "numpy/scipy (BSD) + repo-internal code only (Gen-0)",
191         "wasb_checkpoint_provenance": "C3-UNRESOLVED (ship-blocking; see
COMMERCIALIZATION C3)",
192         "no_paid_llm_training_labels": True,
193         "consent_ledger": "pending-M0.1 (owner-attested: owned footage, all-adult
sessions)",
194     },
195     "gate": verdict,
196 }
197 manifest_out = os.path.join(bundle_dir, "manifest.json")
198 with open(manifest_out, "w", encoding="utf-8") as f:
199     json.dump(manifest, f, indent=2)
200
201     print(f"\n=== Gen-0 artifact bundle: {bundle_dir} ===")
202     print(f"weights : {weights_path}")
203     print(f"manifest: {manifest_out}")
204     print(f"GATE      : ship={verdict['ship']} floor_passed={verdict['floor_passed']} "
205           f"sign_test={verdict['sign_test']}")
206     for b in verdict["blockers"]:
207         print(f"    - {b}")
208     return manifest
209
210
211 def main() -> None:
212     ap = argparse.ArgumentParser(description=__doc__)
213     ap.add_argument("--manifest", required=True,
214                    help="corpus manifest JSON:
[{'name,trajectory,fps,frame_width,labels}, ...]")
215     ap.add_argument("--floor", default=None,
216                    help="floor baseline JSON (e.g.
eval_baselines/heuristic_lovo_n6.json)")
217     ap.add_argument("--out", default=os.path.join("artifacts", "rally_tal_gen0"))
218     ap.add_argument("--version", required=True, help="semver for the artifact bundle")
219     args = ap.parse_args()
220     run(args.manifest, args.out, args.version, floor_path=args.floor)
221
222
223 if __name__ == "__main__":
224     main()
225

```

29. assistant (2026-06-20T16:56:55.798Z)

Now let me check the GCS storage implementation:

30. user (2026-06-20T16:56:56.445Z)

```

1     """Google Cloud Storage backend ? first-class object buckets.
2
3     Uses ``google-cloud-storage``, imported lazily so ``import backend.storage`` never pulls
it;
4     install with the ``storage-gcs`` extra. Credentials come from Application Default
Credentials
5     (``GOOGLE_APPLICATION_CREDENTIALS`` pointing at a service-account JSON, or workload
identity) ?
6     never from config. A ``client`` may be injected for tests.
7     """
8     from __future__ import annotations
9
10    from datetime import timedelta
11    from typing import Any, Iterator, Optional
12

```

```

13     from backend.storage.base import StorageBackend
14     from backend.storage.ref import StorageRef, StorageStat
15
16
17     class GCSStorage(StorageBackend):
18         def __init__(self, *, client: Any = None, project: Optional[str] = None,
19                     signed_url_ttl: int = 3600, **_ignored) -> None:
20             self._ttl = signed_url_ttl
21             if client is not None:
22                 self._client = client
23             return
24             from google.cloud import storage # type: ignore # lazy: only when a gcs:// URI
is used
25             self._client = storage.Client(project=project)
26
27         def _blob(self, ref: StorageRef):
28             return self._client.bucket(ref.bucket).blob(ref.key)
29
30         def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
31             if dst is None:
32                 raise ValueError("GCSStorage.fetch_to_local requires a dst path")
33             import os
34             dst = str(dst)
35             if os.path.exists(dst) and not overwrite:
36                 return dst
37             os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
38             self._blob(ref).download_to_filename(dst)
39             return dst
40
41         def put(self, local_path, ref, *, content_type=None) -> StorageRef:
42             self._blob(ref).upload_from_filename(str(local_path), content_type=content_type)
43             return ref
44
45         def exists(self, ref) -> bool:
46             return bool(self._blob(ref).exists())
47
48         def stat(self, ref) -> StorageStat:
49             b = self._blob(ref)
50             b.reload()
51             return StorageStat(size=int(b.size or 0), modified=b.updated, etag=b.etag,
52                               content_type=b.content_type)
53
54         def list(self, prefix) -> Iterator[StorageRef]:
55             for blob in self._client.list_blobs(prefix.bucket, prefix=prefix.key):
56                 yield StorageRef("gcs", prefix.bucket, blob.name,
f"gcs://{prefix.bucket}/{blob.name}")
57
58         def delete(self, ref) -> None:
59             self._blob(ref).delete()
60
61         def signed_url(self, ref, *, ttl=3600, method="GET") -> str:
62             return self._blob(ref).generate_signed_url(
63                 version="v4", expiration=timedelta(seconds=ttl or self._ttl), method=method)
64

```

31. user (2026-06-20T16:56:56.543Z)

```

1     """Google Drive backend ? reads Drive files through a **locally mounted** Google Drive.
2
3     We make a deliberate, baked-in assumption: **to use Google Drive files you mount Google
Drive
4     for Desktop**, which exposes Drive as an ordinary filesystem path. That's a fair
requirement ? it
5     needs no service account, no API quota, no OAuth, and Drive streams + caches bytes on
access. So

```

```

6      ``gdrive://`` is just a *portable, OS-agnostic alias* over that mount: it names a path
**relative to
7      your "My Drive" folder**, and everything else is plain local-filesystem I/O (the same
code the
8      ``local`` backend uses).
9
10     gdrive://Sharing/clip.mp4  ->  <mount>/My Drive/Sharing/clip.mp4
11
12     Why an alias instead of just a raw path? Portability: the mount root differs per machine
13     (Windows ``G:\\My Drive``, macOS ``~/Library/CloudStorage/GoogleDrive-<acct>/My
Drive``), so the same
14     ``gdrive://`` URI resolves on any box with Drive mounted. The root is **auto-detected**,
or set it
15     explicitly via ``GDRIVE_MOUNT_ROOT`` / ``storage.gdrive.mount_root`` (a path, not a
secret).
16
17     A headless box (a GPU VM, CI) has **no** mount ? there, stage the file to a bucket and
use
18     ``gcs://`` / ``s3://``, or ``gdown <file-id>`` to a local path. This backend
intentionally does NOT
19     talk to the Drive API. When the mount (or a file in it) is missing, errors point the
user at the
20     fix: install Drive for Desktop and include the folder in syncing so it can be pulled
locally.
21     ""
22     from __future__ import annotations
23
24     import glob
25     import os
26     import string
27     from typing import Iterator, Optional
28
29     from backend.storage.base import StorageBackend
30     from backend.storage.local import LocalStorage
31     from backend.storage.ref import StorageRef, StorageStat
32
33     _INSTALL_URL = "https://www.google.com/drive/download/"
34
35     # Shared remedy line ? how to make a folder available locally. Drive for Desktop streams
files,
36     # but a folder only appears once it's part of your synced Drive (and a *shared* folder
must first
37     # be added to "My Drive"). The same fix resolves both "not mounted" and "mounted but
missing".
38     _SYNC_REMEDY = (
39         "make sure the folder is INCLUDED in syncing so its files can be pulled locally: in
Google "
40         "Drive for Desktop keep 'Stream files' on, and for a folder shared with you, open
Drive in the "
41         "browser, right-click it -> 'Add shortcut to Drive' so it appears under 'My Drive'.
Drive then "
42         "downloads the bytes on first access."
43     )
44
45
46     def _not_mounted_msg(tried: Optional[str]) -> str:
47         return (
48             "Google Drive is not mounted, so the gdrive:// backend cannot resolve any
path.\n"
49             f"  Remedy:\n"
50             f"  1. Install Google Drive for Desktop and sign in: {_INSTALL_URL}\n"
51             f"  2. Then {_SYNC_REMEDY}\n"
52             f"  3. If your 'My Drive' is not at the default location (Windows G:\\My Drive,
macOS "
53             "~/Library/CloudStorage/GoogleDrive-<acct>/My Drive), set GDRIVE_MOUNT_ROOT or "

```

```

54         f"storage.gdrive.mount_root to point at it (auto-detect tried: {tried!r})."
55     )
56
57
58     def _not_synced_msg(root: str, rel: str, uri: str) -> str:
59         return (
60             f"Google Drive is mounted ({root!r}) but {uri!r} is not present locally "
61             f"(expected at {os.path.join(root, *rel.split('/')!r}).\n"
62             f" This usually means the folder is not synced yet ? {_SYNC_REMEDY}\n"
63             f" (Install/repair Drive for Desktop if needed: {_INSTALL_URL})"
64         )
65
66
67     def _autodetect_mount_root() -> Optional[str]:
68         """Find a mounted Google Drive 'My Drive' folder, or ``None``. Covers the Google
69 Drive for
70 Desktop defaults: macOS ``~/Library/CloudStorage/GoogleDrive-<acct>/My Drive`` and
71 Windows
72 ``<letter>:\My Drive`` (``G:`` is the install default). Linux/other ? configure
73 explicitly."""
74         home = os.path.expanduser("~")
75         for p in sorted(glob.glob(os.path.join(home, "Library", "CloudStorage",
76 "GoogleDrive*", "My Drive"))):
77             if os.path.isdir(p):
78                 return p
79         if os.name == "nt":
80             # G is the Drive-for-Desktop default; then scan the rest of the assignable
81             letters.
82             for letter in ["G"] + [c for c in string.ascii_uppercase if c != "G"]:
83                 p = f"{letter}:\My Drive"
84                 if os.path.isdir(p):
85                     return p
86         return None
87
88     class GDriveStorage(StorageBackend):
89         def __init__(self, *, mount_root: Optional[str] = None, **_ignored) -> None:
90             root = mount_root or os.environ.get("GDRIVE_MOUNT_ROOT") or
91 _autodetect_mount_root()
92             if not root or not os.path.isdir(root):
93                 raise RuntimeError(_not_mounted_msg(root))
94             self._root = os.path.abspath(root)
95             self._local = LocalStorage(base_dir=self._root) # all I/O is plain local-FS
96 under the mount
97
98         def _local_ref(self, ref: StorageRef) -> StorageRef:
99             """Translate a ``gdrive://<rel>`` ref into a ``local`` ref keyed by an OS path
100 relative to
101 the mount root (``LocalStorage`` joins it under ``base_dir``)."""
102             rel = ref.key.lstrip("/")
103             key = os.path.join(*rel.split("/")) if rel else ""
104             return StorageRef("local", "", key, f"local://{key}")
105
106         def _require_present(self, ref: StorageRef) -> None:
107             """Guard the read paths: a path missing from the mount almost always means the
108 folder
109 isn't synced ? raise with that remedy instead of a cryptic FileNotFoundError
110 downstream."""
111             rel = ref.key.lstrip("/")
112             abs_path = os.path.join(self._root, *rel.split("/")) if rel else self._root
113             if not os.path.exists(abs_path):
114                 raise FileNotFoundError(_not_synced_msg(self._root, rel, ref.uri))
115
116         def _to_gdrive_uri(self, abs_path: str) -> str:
117             """The inverse: an absolute path under the mount ? its ``gdrive://`` URI."""

```

```

109         rel = os.path.relpath(abs_path, self._root).replace(os.sep, "/")
110         return f"gdrive://{rel}"
111
112     def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
113         # The file is already local (via the mount): no dst => identity passthrough to
the mount
114         # path; a dst => copy out (a byte-stable scratch copy), exactly as the local
backend does.
115         self._require_present(ref)
116         return self._local.fetch_to_local(self._local_ref(ref), dst,
overwrite=overwrite)
117
118     def put(self, local_path, ref, *, content_type=None) -> StorageRef:
119         self._local.put(local_path, self._local_ref(ref), content_type=content_type)
120         return ref
121
122     def exists(self, ref) -> bool:
123         return self._local.exists(self._local_ref(ref))
124
125     def stat(self, ref) -> StorageStat:
126         self._require_present(ref)
127         return self._local.stat(self._local_ref(ref))
128
129     def list(self, prefix) -> Iterator[StorageRef]:
130         for r in self._local.list(self._local_ref(prefix)):
131             uri = self._to_gdrive_uri(r.key)
132             yield StorageRef("gdrive", "", uri[len("gdrive://"):], uri)
133
134     def delete(self, ref) -> None:
135         self._local.delete(self._local_ref(ref))
136

```

32. assistant (2026-06-20T16:56:59.154Z)

Now let me search for any other artifact write operations:

33. user (2026-06-20T16:57:00.142Z)

```

run_regression.py:79:         json.dump(result.as_dict(), f, indent=2)
run_eval.py:123:         json.dump(report_to_dict(report), f, indent=2)
training\gen0\harness.py:162:         json.dump(payload, f, indent=2)
training\gen0\harness.py:199:         json.dump(manifest, f, indent=2)
scripts\run_phase3_eval.py:172:         json.dump({"segmenter": args.segmenter, "iou":
args.iou, "stages": stages,
artifacts\so.py:56:         json.dump(obj, f)
scripts\nightly_regression.py:543:         json.dump(out, fh, indent=2, sort_keys=True)
scripts\nightly_regression.py:614:         json.dump({"date": date, "status":
result.overall_status(),
backend\eval\ablation.py:485:         json.dump(report.as_dict(), f, indent=2)
backend\eval\annotations.py:63:         writer = csv.writer(f)
backend\worker\_main__.py:62: def _write_intervals_csv(segments: List[dict], path: str) -> None:
backend\worker\_main__.py:66:         w = csv.writer(f)
backend\worker\_main__.py:77: def _write_report_json(video_id: str, segments: List[dict],
status: str, path: str) -> None:
backend\worker\_main__.py:79:         json.dump({
backend\worker\_main__.py:146:         _write_intervals_csv(segments, os.path.join(out_local,
"intervals.csv"))
backend\worker\_main__.py:147:         _write_report_json(video_id, segments, status,
os.path.join(out_local, "report.json"))
backend\worker\_main__.py:316:         json.dump(specs, f, indent=2)
tests\test_config.py:43:         json.dump({"indexing": {"skip_ai_handoff": True}}, fh)
tests\test_config.py:52:         json.dump({"logging": {"verbose_http": True}}, fh)
tests\test_config.py:81:         json.dump({"indexing": {"fusion": {"min_crossings": 2}}},
fh)
tests\test_config.py:100:         json.dump({"indexing": {"fusion": {

```

```

tests\test_config.py:127:         json.dump(data, f)
tests\test_config.py:148:         json.dump(data, f)
tests\test_config.py:241:         json.dump({"output": {"output_dir": "reels", "db_name":
"x.db"}}, f)
tests\test_config.py:255:         json.dump({"indexing": {"default_segmenter": "motion"}},
fh)
tests\test_config.py:269:         json.dump({
tests\test_audio_features.py:51:             json.dump(data, f)
tests\test_audio_features.py:64:             json.dump(data, f)
tests\test_audio_features.py:100:             json.dump(data, f)
tests\test_calibrate_wasb.py:20:         csv.writer(f).writerows(rows)
tests\test_calibrate_wasb.py:26:         w = csv.writer(f)
backend\eval\classifier.py:109:             json.dump(self.to_dict(), f, indent=2)
tests\test_calibrate_local.py:16:         csv.writer(f).writerows(rows)
tests\test_fusion_golden_telemetry.py:52:         json.dump(manifest, f)
tests\test_ablation.py:354:         json.dump(data, f)
tests\test_ablation.py:370:         json.dump(data, f)
tests\test_ablation.py:489:         json.dump(data, f)
tests\test_eval_partial_labels.py:23:         csv.writer(f).writerows(rows)
tests\test_eval_partial_labels.py:30:         w = csv.writer(f)
tests\test_distill_local.py:17:         csv.writer(f).writerows(rows)
tests\test_fusion_compare.py:30:         json.dump(data, f)
tests\test_fusion_compare.py:43:         json.dump(data, f)
tests\test_fusion_compare.py:144:         json.dump(manifest, f)
tests\test_fusion_compare.py:147:         json.dump({"name": "alpha", "fps": 30.0, "frame_width":
1920.0, "candidates": [], "gts": []}, f)
tests\test_fusion_compare.py:178:         json.dump(manifest, f)
tests\test_export_wasb_segments.py:18:         csv.writer(f).writerows(rows)
tests\test_export_wasb_segments.py:24:         w = csv.writer(f)
tests\test_experiment.py:84:         json.dump(data, f)
tests\test_gt_loader.py:22:         w = csv.writer(f)
tests\test_gt_loader.py:32:         w = csv.writer(f)
backend\pipeline\detectors\stub_runner.py:112:         w = _csv.writer(f)
backend\pipeline\detectors\stub_runner.py:148:         w = _csv.writer(f)
backend\pipeline\detectors\wasb_infer.py:209:         w = csv.writer(f)
backend\eval\fusion_golden.py:169:         json.dump(data, f, indent=2)
backend\eval\fusion_audio.py:48:         json.dump(data, f, indent=2)
backend\eval\experiment.py:618:         json.dump(entries, f, indent=2)
backend\eval\rally_seg_eval.py:549:         json.dump(result, fh, indent=2)
backend\eval\serve_contrast.py:162:         json.dump(result, f, indent=2)
backend\eval\per_user_regression.py:157:         json.dump(baselines, f, indent=2,
sort_keys=True)
backend\eval\regression.py:98:         json.dump(baselines, f, indent=2, sort_keys=True)

```

[Showing results with pagination = limit: 60]

34. assistant (2026-06-20T16:57:02.300Z)

Now let me look at more storage operations and check how the database gets written:

35. user (2026-06-20T16:57:02.982Z)

grep: C:\Users\avidu\Projects\badminton-highlight-indexer\backend: No such file or directory

36. user (2026-06-20T16:57:06.174Z)

Found 1 file
backend\infrastructure\database.py

37. user (2026-06-20T16:57:08.241Z)

```

1     import sqlite3
2     import json
3     import logging

```

```

4     from contextlib import contextmanager
5     from typing import List, Dict, Any, Optional, Tuple
6
7     logger = logging.getLogger(__name__)
8
9     class Database:
10         def __init__(self, db_path: str):
11             self.db_path = db_path
12             self._init_db()
13
14         @contextmanager
15         def _connect(self):
16             """Transaction scope that also CLOSES the connection.
17
18             ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
19             never closes, so every call leaked its connection to GC (lingering file
20             handles on the WAL sidecars under Windows). Internal methods use this.
21             """
22             conn = self._get_connection()
23             try:
24                 with conn:
25                     yield conn
26             finally:
27                 conn.close()
28
29         def _get_connection(self):
30             # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
31             # adds concurrent writers (a swallowed 'database is locked' silently drops
32             segments).
33             conn = sqlite3.connect(self.db_path, timeout=30.0)
34             conn.row_factory = sqlite3.Row
35             # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
36             conn.execute("PRAGMA foreign_keys = ON")
37             # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
38             conn.execute("PRAGMA busy_timeout = 30000")
39             try:
40                 conn.execute("PRAGMA journal_mode = WAL")
41             except sqlite3.OperationalError:
42                 pass # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
43             return conn
44
45         def _execute_write(self, sql: str, params: Tuple[Any, ...], error_msg: str) -> bool:
46             """Run ONE bool-returning single-statement write inside a connection scope.
47
48             Captures the connect/commit/except-logger.error/return-False boilerplate shared
49             by
50
51             the bool-returning single-statement writers. ``error_msg`` is the caller's exact
52             per-method message (already formatted with its own ids) ? logged verbatim on a
53             swallowed write fault so each writer keeps its specific log text. Returns True
54             on
55
56             success, False (after logging ``error_msg``) on any exception."""
57             try:
58                 with self._connect() as conn:
59                     conn.cursor().execute(sql, params)
60                     conn.commit()
61                 return True
62             except Exception as e:
63                 logger.error(f"{error_msg}: {e}")
64                 return False
65
66         @staticmethod
67         def _add_column_idempotent(cursor: sqlite3.Cursor, ddl: str) -> None:
68             """Apply a bare ``ALTER TABLE ... ADD COLUMN`` migration step re-runnably.
69
70             A DDL ``ALTER`` auto-commits independently of the surrounding ``with conn:``

```

```

66         transaction, so an interrupted v3/v4 block can leave the column added while
67         ``user_version`` stays un-bumped ? and the next open would re-run the same
68         ``ALTER`` and raise ``OperationalError: duplicate column name``, bricking every
69         future ``Database(...)`` construction (``_init_db`` runs in ``__init__``).
Swallow
70     ONLY that specific error so the ladder stays crash-safe, matching the re-
runnable
71     ``CREATE ... IF NOT EXISTS`` v1/v2/v5 blocks.
72     """
73     try:
74         cursor.execute(ddl)
75     except sqlite3.OperationalError as exc:
76         if "duplicate column name" not in str(exc).lower():
77             raise
78
79     def _init_db(self):
80         """Initializes tables for videos, play_segments, and events."""
81         with self._connect() as conn:
82             cursor = conn.cursor()
83
84             self._create_base_tables(cursor)
85
86             # Versioned migrations live here. PRAGMA user_version is the schema
87             # contract ? bump it for every change and add an ordered `if version < N`
88             # block below. Tests assert the expected version, so accidental drift fails
89             version = cursor.execute("PRAGMA user_version").fetchone()[0]
90
91             if version < 1:
92                 self._migrate_to_v1(cursor)
93
94             if version < 2:
95                 self._migrate_to_v2(cursor)
96
97             if version < 3:
98                 self._migrate_to_v3(cursor)
99
100             if version < 4:
101                 self._migrate_to_v4(cursor)
102
103             if version < 5:
104                 self._migrate_to_v5(cursor)
105             # future: if version < 6: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 6
106
107         conn.commit()
108
109     def _create_base_tables(self, cursor):
110         """Create the videos / play_segments / events base tables (idempotent)."""
111         # Videos Table
112         cursor.execute("""
113             CREATE TABLE IF NOT EXISTS videos (
114                 id TEXT PRIMARY KEY,
115                 filepath TEXT NOT NULL,
116                 processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
117                 status TEXT NOT NULL,
118                 validation_results TEXT
119             )
120         """)
121
122         # Play Segments Table (Extensible metadata JSON)
123         cursor.execute("""
124             CREATE TABLE IF NOT EXISTS play_segments (
125                 id INTEGER PRIMARY KEY AUTOINCREMENT,
126                 video_id TEXT NOT NULL,

```

```

127         start_time REAL NOT NULL,
128         end_time REAL NOT NULL,
129         duration REAL NOT NULL,
130         server TEXT,
131         receiver TEXT,
132         metadata TEXT,
133         FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
134     )
135     """
136
137     # Events Table
138     cursor.execute("""
139         CREATE TABLE IF NOT EXISTS events (
140             id INTEGER PRIMARY KEY AUTOINCREMENT,
141             segment_id INTEGER NOT NULL,
142             timestamp REAL NOT NULL,
143             type TEXT NOT NULL,
144             player TEXT,
145             details TEXT,
146             FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE CASCADE
147         )
148     """)
149
150     def _migrate_to_v1(self, cursor):

```

38. assistant (2026-06-20T16:57:10.785Z)

Perfect! Now I have a comprehensive understanding. Let me create a structured summary by mapping all write sites:

39. user (2026-06-20T16:57:36.768Z)

Structured output provided successfully

40. assistant (2026-06-20T16:57:44.516Z)

Perfect! I've completed a comprehensive mapping of all per-video artifact write sites in the WORKER + STORAGE + BUCKET layout. Here's the summary of my findings:

Key Artifacts Mapped

****cmd_infer Output** (per-video inference)**

- ****intervals.csv****: `{out\_uri}/outputs/{video\_id}/intervals.csv` - Rally boundaries + metadata
- ****report.json****: `{out\_uri}/outputs/{video\_id}/report.json` - Status + segment summary
- All artifacts keyed by `video\_id` (MD5 hash of video bytes, line 102)

****cmd_train Output** (training artifacts)**

- ****weights/{version}/weights.json****: Logistic regression model + calibration (threshold, smoothing params)
- ****weights/{version}/manifest.json****: Complete training lineage, per-video GT hashes, LOVO eval results, ship-gate verdict
- Keyed by `--version` semver argument (line 322, 338)

****Local Scratch Artifacts** (not pushed to bucket)**

- ****sports_indexer.db****: SQLite3 database stored in scratch (line 107) - NOT pushed, stays local
- ****in.mp4****: Downloaded video temp copy (line 98)
- ****labels/, trajectories/, videos/****: cmd_train working directories (lines 249-251, 282-289)
- ****manifest.json****: Corpus training spec before harness subprocess (line 316)

Storage Routing

All `storage.put()` and `storage.fetch()` calls respect the pluggable backend:

- ****local****: Identity passthrough (bare paths or `local://`)

- **s3**: Download/upload via boto3 (supports AWS S3, R2, DigitalOcean Spaces, MinIO)
- **gcs**: Google Cloud Storage via google-cloud-storage SDK
- **gdrive**: Locally-mounted Google Drive (Drive for Desktop)

Config knobs: ``config.storage.backend``, ``config.storage.{s3,gcs,gdrive}.*`` sections + environment overrides (`AWS_ENDPOINT_URL_S3`, `GOOGLE_APPLICATION_CREDENTIALS`, `GDRIVE_MOUNT_ROOT`).

All write operations go through ``storage.put()`` (line 153, 339) which routes to the backend's implementation (`S3Storage.put`, `GCSStorage.put`, `LocalStorage.put`, or `GDriveStorage.put`).